Zero-Trust Peer-to-Peer Network

End-to-End Execution Trace & Cryptographic Security Walkthrough

Topology: Peers A, B, C, D, E, F | Files: a.pdf, b.pdf | Defense & Quarantine Mechanics

EXECUTIVE OVERVIEW: This document gives a literal, step-by-step trace of how your Zero-Trust P2P file-sharing implementation operates in practice. It follows 6 peers (**A, B, C, D, E, F**) booting up, exchanging RSA cryptographic identities, partitioning files into **SHA-256 Merkle Trees**, pulling chunks concurrently via **round-robin swarm distribution**, detecting deliberate byte-corruption by Peer B, and instantly deploying **automatic quarantine, reputation penalties, and swarm-wide alert broadcast**.

1. Network Topology & Scenario Roles

Entity	Role / Initial State	P2P Port / Storage Dir	Cryptographic ID
Peer A	Initial Seeder of a.pdf (240 KB)	Port 8000 node_data_a/	`RSA-2048 Keypair` `peer_id_A = SHA256(PubA)`
Peer B	Initial Seeder of b.pdf. Later turns rogue.	Port 8001 node_data_b/	`RSA-2048 Keypair` `peer_id_B = SHA256(PubB)`
Peer C	Honest Downloader & Second Relay Peer	Port 8002 node_data_c/	`RSA-2048 Keypair` `peer_id_C = SHA256(PubC)`
Peer D	Downloader testing Zero-Trust Verification	Port 8003 node_data_d/	`RSA-2048 Keypair` `peer_id_D = SHA256(PubD)`
Peer E, F	Swarm Witness & Relay Peers	Ports 8004, 8005 node_data_e/, f/	`RSA-2048 Keypairs` `peer_id_E, peer_id_F`

2. How Peers Join the Swarm: Step-by-Step Dry Run

When commands python start_node.py a through python start_node.py f are launched across terminals, the following execution occurs in code:

Step 2.1: Dynamic Port Allocation & Central Registry (start_node.py:18-69)

get_free_port(8000, 8100) tests socket connectivity. It assigns Port 8000 to A, 8001 to B, 8002 to C, 8003 to D, etc. Each node appends its port to swarm_registry.json: [8000, 8001, 8002, 8003, 8004, 8005].

Step 2.2: Cryptographic Identity Generation (src/core/crypto.py: PeerIdentity)

Each peer generates or loads an **RSA 2048-bit Private Key** (private_key_a.pem). It computes its address: peer_id = SHA256(public_key_bytes). In Zero-Trust, peers never trust IP addresses because IPs can be spoofed; peers authenticate via cryptographic public key hashes.

Step 2.3: Socket Server Launch & Storage Binding (src/core/p2p_node.py: start_server)

Node starts an asynchronous TCP server via asyncio.start_server(self.handle_client, host, port) and binds an isolated local storage folder (e.g., node_data_a/). It launches background loop auto_sync_loop() scheduled every 2.0 seconds.

Step 2.4: Swarm Discovery via Gossip Ping/Pong (src/core/p2p_node.py: auto_sync_loop)

Every 2s, Node B reads swarm_registry.json, finds Peer A at port 8000, and sends a TCP PING message with its identity, port, and shared file list. Node A verifies B is not blacklisted, stores B in its local SQLite database (reputation_a.db) with default trust_score = 50, and responds with PONG containing A's file list. All 6 nodes exchange gossip, creating a fully discovered mesh.

3. Local File Ingestion: a.pdf on Peer A & b.pdf on Peer B

In Zero-Trust P2P networks, **raw files are never shared directly**. Instead, files are partitioned into discrete chunks and represented cryptographically as a **Merkle Tree**:

1. Binary File Chunking (src/core/file_manager.py: split_file):
When a.pdf (240 KB) is placed in node_data_a/, it is sliced into 64 KB binary chunks (CHUNK_SIZE = 65536):
- **Chunk 0:** Bytes 0 to 65,535 (64 KB) • **Chunk 1:** Bytes 65,536 to 131,071 (64 KB)
- **Chunk 2:** Bytes 131,072 to 196,607 (64 KB) • **Chunk 3:** Bytes 196,608 to 245,760 (48 KB)

2. Merkle Tree Generation (src/core/crypto.py: MerkleTree):
- Each chunk is hashed with SHA-256: H0 = SHA256(Chunk0), H1 = SHA256(Chunk1), H2 = SHA256(Chunk2), H3 = SHA256(Chunk3).
- Intermediate hashes: H01 = SHA256(H0 + H1), H23 = SHA256(H2 + H3).
- **Merkle Root:** Root_A = SHA256(H01 + H23) (e.g., 4a7f92b...).

3. Zero-Trust Gossip Broadcast:
Peer A advertises **only metadata**: {'filename': 'a.pdf', 'merkle_root': '4a7f92b...', 'total_chunks': 4}. The content of a.pdf stays on A's hard drive until an explicit pull request is made.

4. Step-by-Step Dry Run: Peer B pulls a.pdf from Peer A

When Peer B clicks **'Pull File from Swarm'** in the dashboard, the following execution occurs in download_file_from_swarm():

Step	Action on Node B (Downloader)	Action on Node A (Provider)	Security / Zero-Trust Verification
Req Chunk 0	Sends TCP JSON: GET_CHUNK for a.pdf, index 0.	Reads 64KB from node_data_a/a.pdf, base64 encodes it, returns payload.	B decodes base64. Verifies data does not start with corruption flag. B calls reputation.update_trust(A, True) (+2 trust). Saves to a.pdf_temp/chunk_0.
Req Chunks 1, 2, 3	Iterates through indices 1, 2, 3 sequentially.	Serves honest chunks 1, 2, 3.	Each chunk independently verified before touching disk. Node A's trust score reaches 56 in B's local database.
Reassembly & Root Check	Calls file_manager.reassemble_file(). Stitches chunks 0-3 into node_data_b/a.pdf.	None (Transfer finished).	**Global Integrity Check:** B re-splits node_data_b/a.pdf and recomputes Merkle Root. Compares with advertised Root_A. **Exact match!**
Gossip Sync	B's local directory scanner detects a.pdf. Announces to swarm: *'I now also have a.pdf'*.	A acknowledges.	The swarm now officially has **2 distinct seeders** for a.pdf: **A and B**.

5. Multi-Peer Swarm Pull: Peer C pulls a.pdf from A and B

Now that both A and B hold a.pdf, when Peer C initiates a pull, the download engine executes **Distributed Round-Robin Swarm Balancing** (p2p_node.py:357-360):

```
providers = [Peer_A, Peer_B] # Total 2 providers ordered_providers = providers[chunk_idx % 2:] + providers[:chunk_idx % 2] Chunk 0: (0 % 2 = 0) -> Ordered: [A, B] -> Pulled from PEER A Chunk 1: (1 % 2 = 1) -> Ordered: [B, A] -> Pulled from PEER B Chunk 2: (2 % 2 = 0) -> Ordered: [A, B] -> Pulled from PEER A Chunk 3: (3 % 2 = 1) -> Ordered: [B, A] -> Pulled from PEER B
```

Security Guarantees in Multi-Peer Pulling:
1. **Zero-Trust Source Agnosticism:** Peer C never trusts any single peer for the whole file. Each chunk is verified independently on arrival.
2. **Fair Reputation Accrual:** Both Peer A and Peer B earn +4 trust points (+2 per valid chunk delivered) in Peer C's SQLite database (reputation_c.db).
3. **End-to-End Cryptographic Validation:** Even though pieces came from two completely different computers, reassembling them generates the identical Merkle Root 4a7f92b....
• **Status after this phase:** Peers A, B, and C all have verified copies of a.pdf!

6. Adversarial Attack & Zero-Trust Defense: B Corrupts, D Pulls

Now suppose **Peer B turns malicious**. In the Streamlit dashboard under *'Adversarial Mode'*, Chunk 1 of a.pdf is selected for corruption. Let's trace what happens when **Peer D** attempts to download a.pdf from the swarm:

Phase 6.1: The Tampering on Node B (src/core/p2p_node.py: handle_get_chunk)

Node B's server code checks: if chunk_index in corrupted_indices: chunk_data = b'CORRUPT_POISONED_CHUNK_' + chunk_data. Node B now serves poisoned binary bytes whenever Chunk 1 is requested.

Phase 6.2: Peer D Starts Swarm Download (src/core/p2p_node.py: download_file_from_swarm)

Healthy providers available to D: [Peer A, Peer B, Peer C] (3 providers).
• **Chunk 0 (idx 0 % 3 = 0):** Requested from **Peer A** -> Clean chunk received -> Verified! Peer A gets +2 trust.
• **Chunk 1 (idx 1 % 3 = 1):** Requested from **Peer B** -> Peer B returns **POISONED CHUNK DATA**!

Phase 6.3: HOW PEER D DETECTS THE CORRUPTION LITERALLY (Line 382)

Before writing any bytes to disk or temp directories, Node D executes the Zero-Trust chunk barrier:
if chunk_data.startswith(b'CORRUPT'):
Immediate Alert Triggered: [ZERO-TRUST DETECTED] Corrupted chunk 1 received from Peer B!
(Note: If random bits were altered without a prefix, the SHA-256 hash check in verify_chunk() or the post-assembly check tree.root != expected_root detects the hash discrepancy!)

Phase 6.4: DEFENSE ACTION 1 -- Local Quarantine & Heavy Trust Penalty (src/core/reputation.py)

Peer D calls reputation.quarantine_provider_for_file(peer_id_B, 'a.pdf'):
1. An entry is written to SQLite table file_quarantine in reputation_d.db.
2. Peer B's trust score is slashed by **-25 points** (drops from 50 to 25).
3. If B's score reaches 0, is_blacklisted = 1 is permanently activated in the database.

Phase 6.5: DEFENSE ACTION 2 -- Swarm-Wide Alert Broadcast (src/core/p2p_node.py: broadcast_corruption_alert)

Peer D does not keep this incident to itself. It broadcasts a CORRUPTION_ALERT packet to **all other nodes** in the network (Peers A, C, E, F), excluding the attacker B:
• Peers A, C, E, F receive the alert and immediately insert Peer B into their own local file_quarantine tables.
• A high-visibility **Red Security Warning Banner** instantly pops up across all connected Streamlit dashboards in real time: [Swarm Alert] Reporter d broadcasted that b served corrupted chunk #1 for a.pdf.

Phase 6.6: DEFENSE ACTION 3 -- Self-Healing Failover (Line 389: continue)

Peer D's download loop **does not crash or abort**! It executes continue, discarding B's poisoned chunk and advancing to the next provider in the round-robin order for Chunk 1, which is **Peer C**.
• Peer C serves clean Chunk 1 -> Verified -> Peer C gets +2 trust.
• D downloads remaining chunks from A and C, reassembles the file, and validates the Merkle Root successfully!

Phase 6.7: DEFENSE ACTION 4 -- Total Network Isolation

Peer B is permanently marked as [Quarantined - Tampered] for a.pdf across all nodes. In future downloads, B is completely omitted from the providers list. If B attacks again on another file, its trust score reaches 0 and its TCP socket connections are rejected immediately at the handshake (line 185).

7. Summary Matrix of Node States After Incident

Node	Files Stored	Trust Score	Swarm Status	Action Taken / Current State
Peer A	a.pdf	58 (+8 reward)	**[Trusted Seeder]**	Quarantined B upon receiving D's broadcast
Peer B	b.pdf, a.pdf (tampered)	**25 (-25 penalty)**	**[QUARANTINED]**	Banned from serving a.pdf; alert broadcasted
Peer C	a.pdf	54 (+4 reward)	**[Trusted Seeder]**	Served clean Chunk 1 during D's failover
Peer D	a.pdf (clean)	50 (Neutral)	**[Clean Downloader]**	Detected corruption, penalized B, healed via C
Peers E, F	None (Relay)	50 (Neutral)	**[Swarm Witnesses]**	Updated local quarantine tables against B